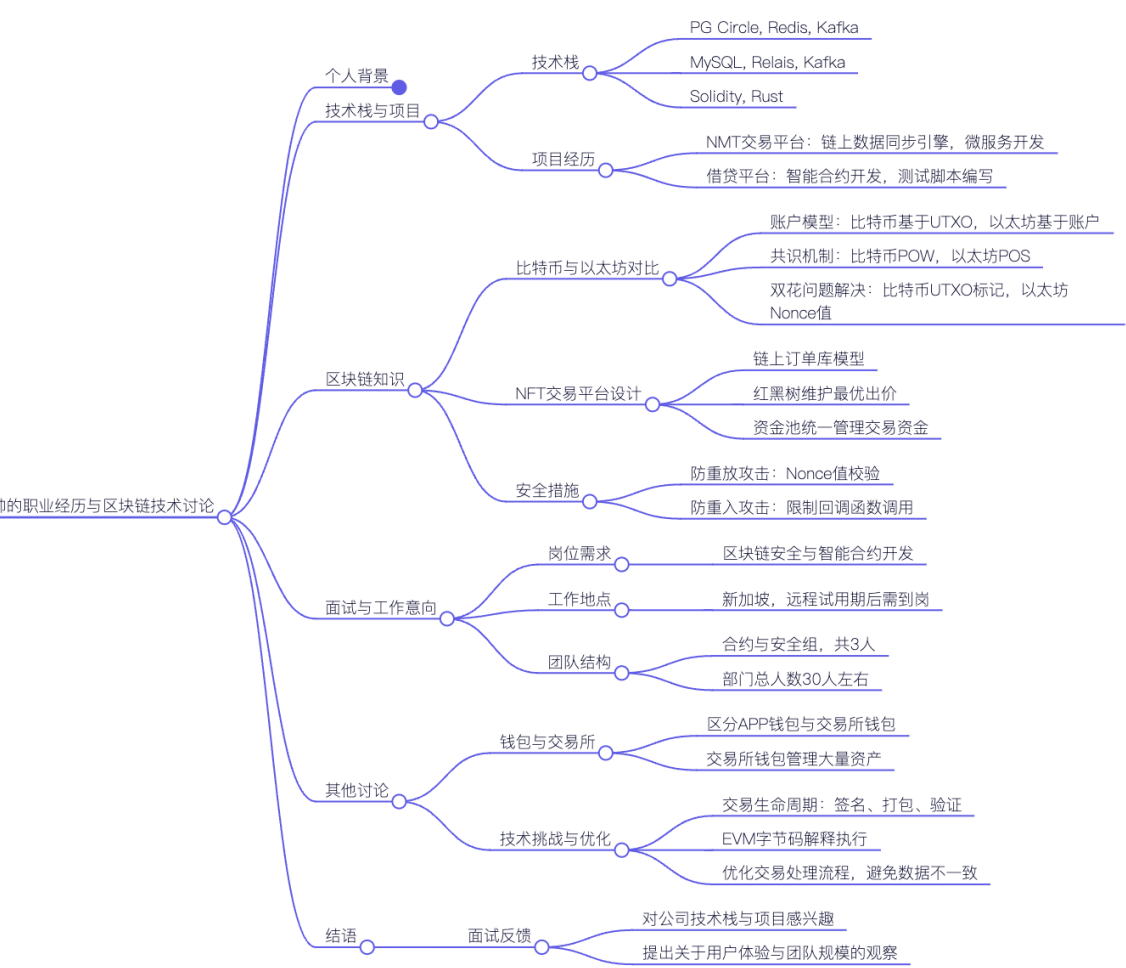


Bitget 面试题回顾（整理版）

关键技术栈

后端开发 · 计算机工程 · 服务器开发 · 链上数据同步 · 智能合约 · EVM · 比特币 · 以太坊 · UTXO · 权益证明 · NFT交易 · 资金池 · 撮合机制 · 版税收益 · 安全 · 重入攻击 · 合约安全 · 签名验证 · 交易机制

思维导图



一、区块链基础知识

1.1 比特币与以太坊的核心区别

面试官：你目前主要接触的是 EVM 吗？能否讲一下比特币和以太坊的区别？

回答：

目前主要接触的是 EVM 生态。比特币与以太坊的主要区别体现在以下两个方面：

账户模型：

- 比特币：采用 UTXO（未花费的交易输出）模型。每次交易会消耗旧的 UTXO 并生成新的 UTXO，用户余额通

过统计所有未花费的 UTXO 计算得出

- **以太坊**：采用账户余额模型。每个账户地址直接记录余额，交易时余额直接增减，类似传统银行账户

共识机制：

- **比特币**：一直采用工作量证明（PoW）机制
- **以太坊**：已从 PoW 转向权益证明（PoS）机制。PoS 机制下，持有更多代币的节点拥有更高的出块权重，但小资本质押者理论上也有机会参与验证和获得收益

1.2 以太坊 Nonce 值的作用

面试官：以太坊如何通过 nonce 值解决交易重复的问题？

回答：

以太坊为每个账户维护一个递增的 nonce 值来防止交易重放：

- 每个账户的 nonce 从 0 开始，每发送一笔交易递增 1
- 每笔交易必须携带正确的 nonce 值
- 节点会验证交易的 nonce 是否等于账户当前 nonce
- 如果提交重复 nonce 的交易，节点会拒绝处理，从而防止交易重放攻击

1.3 交易加速机制

面试官：当比特币交易卡住时，如何加速交易过程？

回答：

以太坊：

可以通过以下方式加速交易：

- 使用相同 nonce 发送新交易，但提高 Gas Price
- 矿工会优先打包 Gas Price 更高的交易

比特币：

常见的加速方法包括：

- **RBF (Replace-By-Fee)**：替换原交易并提高手续费
- **CPFP (Child Pays For Parent)**：通过子交易支付更高手续费，激励矿工同时打包父交易

二、NFT 交易市场项目

2.1 项目架构与团队

面试官：在 NFT 交易市场项目中，合约团队的规模及整体项目架构是怎样的？NFT 交易市场的交易撮合机制是如何设计的？

回答：

团队规模：

- 合约团队共 12 人
- 7 人负责后端服务开发
- 5 人负责智能合约开发

项目架构：

采用链上订单簿模型设计 NFT 交易系统，核心功能包括：

- NFT 铸造
- 上架卖单/买单
- 订单撮合机制
- 版税功能
- 资金池管理

撮合机制设计：

1. **买方订单管理：**维护一个优先出价队列（使用红黑树数据结构），按价格从高到低记录所有买家出价
2. **最优价格展示：**为卖家实时展示当前最高买入价
3. **撮合流程：**
 - 卖家选择心仪的买单
 - 调用合约 `match` 函数进行链上撮合
 - 合约验证：NFT 所有权、价格合法性、买家余额充足性
 - 验证通过后，从资金池转移 NFT 给买家，转移资金给卖家

2.2 版税功能实现

面试官：NFT 交易市场中的版税功能如何在合约层面实现？为何选择通过资金池合约进行所有交易交割？

回答：

版税功能实现：

- 每个 NFT 系列可设置不同的版税率
- 热门且流动性高的 NFT 系列可收取更高版税
- **实现机制：**
 - NFT 上架时，预先扣留上架价格的一定比例作为版税
 - 版税资金锁定在资金池合约中
 - 交易成功后，版税自动转移到版税收益合约
 - 创作者可随时提取版税收益

资金池合约的优势：

1. **资金可控性：**所有资产（用户代币、ETH）统一管理
 2. **交易实时性：**无需等待用户钱包授权，减少交易延迟
 3. **用户体验：**网络拥堵时也能保证快速交割
 4. **便于收费：**方便收取平台手续费和版税
 5. **安全性：**集中管理便于实施安全措施
-

2.3 资金流处理

面试官：在订单匹配过程中，为什么选择将所有资金通过你们的合约进行处理，而不是直接从用户钱包转账？

回答：

选择资金池模式的核心原因：

1. **保证实时性**：直接从用户钱包转账需要等待链上确认，网络拥堵时延迟严重
 2. **提升用户体验**：用户提前充值到资金池，交易时即时完成
 3. **便于收费**：统一的资金流便于扣除手续费和版税
 4. **降低 Gas 成本**：批量处理降低单笔交易的 Gas 消耗
-

2.4 买卖双方交易流程

面试官：能否详细描述一下作为买家和卖家，在合约中是如何处理资金流的？

回答：

卖家流程：

1. 调用 `list` 函数上架 NFT
2. 授权合约管理该 NFT（`approve` 或 `setApprovalForAll`）
3. 设定售价和版税比例
4. 链上记录卖单信息

买家流程：

1. 浏览市场或创建买单
2. 买单会记录在链上订单簿中
3. 点击购买后，前端发送交易到合约
4. 合约验证：
 - 买家余额是否充足
 - NFT 是否仍在售
 - 价格是否匹配

撮合与交割：

1. 验证通过后执行撮合
 2. NFT 从卖家转移给买家
 3. 资金流转：
 - 扣除平台手续费
 - 扣除版税（转给创作者）
 - 剩余资金转给卖家
-

2.5 对比优势

面试官：相较于其他 NFT 平台，你们的模式有哪些优势？

回答：

相比其他 NFT 平台，我们的优势包括：

1. **高效的价格发现**：直接展示最优出价，买卖双方快速匹配
2. **更快的撮合速度**：链下筛选订单，链上只执行确定的撮合，减少计算复杂度
3. **降低 Gas 成本**：避免在合约中进行复杂的订单筛选逻辑
4. **更好的用户体验**：实时显示最佳价格，交易确定性高

三、安全机制

3.1 重入攻击防范

面试官：你们是否参考了 OpenSea 方案？在安全方面，特别是撤单时如何防范重入攻击等安全问题？

回答：

参考方案：

我们参考了 OpenSea 的设计方案，但在安全方面做了额外加固。

防重入攻击措施：

1. **遵循 CEI 模式** (Checks-Effects-Interactions)：
 - 先进行检查 (Checks)
 - 再修改状态 (Effects)
 - 最后进行外部调用 (Interactions)
2. **撤单流程示例**：

```
// 1. 检查订单状态
require(orders[orderId].isActive, "Order not active");

// 2. 修改状态（防重入的关键）
orders[orderId].isActive = false;

// 3. 转账（外部调用放最后）
token.transfer(msg.sender, amount);
```

3. **使用重入锁 (ReentrancyGuard)**：为关键函数添加 `nonReentrant` 修饰符

3.2 重入攻击原理

面试官：重入攻击是如何发生的？对于有重入漏洞的合约，攻击者如何实现攻击？

回答：

重入攻击原理：

当合约在转账（外部调用）前未更新状态时，攻击者可以在接收资金的回调函数中再次调用原函数，重复提取资金。

攻击示例：

易受攻击的合约：

```
function withdraw(uint amount) public {
    require(balances[msg.sender] >= amount);

    // 危险：先转账，后更新状态
    (bool success, ) = msg.sender.call{value: amount}("");
    require(success);

    balances[msg.sender] -= amount; // 太晚了!
}
```

攻击者合约：

```
contract Attacker {
    VulnerableContract target;

    fallback() external payable {
        // 在接收资金时再次调用 withdraw
        if (address(target).balance >= 1 ether) {
            target.withdraw(1 ether);
        }
    }

    function attack() external payable {
        target.withdraw(1 ether); // 发起攻击
    }
}
```

攻击流程：

1. 攻击者调用 `withdraw`
2. 合约转账触发攻击者的 `fallback` 函数
3. `fallback` 中再次调用 `withdraw`
4. 因为余额尚未更新，检查通过
5. 重复步骤 2-4，直到合约余额耗尽

3.3 Receive 与 Fallback 函数

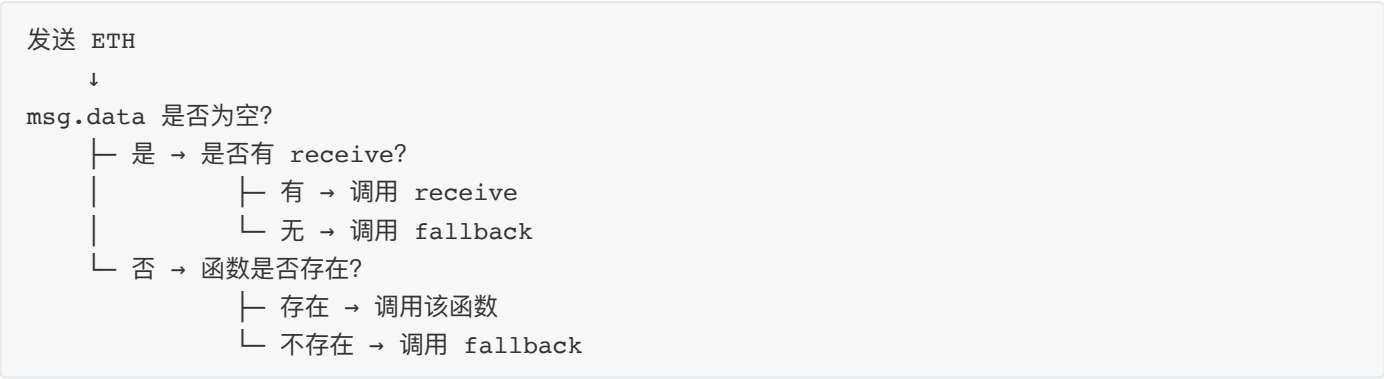
面试官：receive 和 fallback 函数有什么区别？

回答：

函数特性对比：

特性	receive	fallback
触发条件	接收 ETH 且 calldata 为空	调用不存在的函数 或 接收 ETH 但无 receive
参数	无参数	<code>fallback(bytes calldata) returns (bytes memory)</code>
主要用途	明确的收款行为	兜底机制
优先级	高	低

调用优先级：



示例：

```
contract Example {
    event Received(address sender, uint amount);
    event Fallback(address sender, uint amount, bytes data);

    receive() external payable {
        emit Received(msg.sender, msg.value);
    }

    fallback() external payable {
        emit Fallback(msg.sender, msg.value, msg.data);
    }
}
```

四、签名验证与安全

4.1 签名校验的目的

面试官：签名校验的主要目的是什么？

回答：

签名校验的核心目的：

- 1. 身份验证：确认交易确实由私钥持有者发起
- 2. 防止机器人攻击：确保交易由真实用户签名
- 3. 提高成功率：前端预先验证签名合法性，避免无效交易上链浪费 Gas

4. 授权确认：某些操作（如大额转账）需要用户明确签名授权

4.2 防止签名重放攻击

面试官：如何防止签名重放攻击？重放攻击是什么，以及如何预防？

回答：

什么是重放攻击：

攻击者截获一次有效签名后，在其他场景重复使用该签名执行未经授权的操作，导致用户资产损失。

防护措施：

1. 使用 Nonce 值：

```
mapping(address => uint256) public nonces;

function executeWithSignature(
    address user,
    uint256 amount,
    uint256 nonce,
    bytes memory signature
) external {
    require(nonce == nonces[user], "Invalid nonce");

    bytes32 hash = keccak256(abi.encodePacked(user, amount, nonce));
    require(recoverSigner(hash, signature) == user, "Invalid signature");

    nonces[user]++; // 使用后递增

    // 执行操作...
}
```

2. 包含合约地址：

防止签名在不同合约间重放

```
bytes32 hash = keccak256(abi.encodePacked(
    address(this), // 合约地址
    chainId,       // 链 ID
    user,
    amount,
    nonce
));
```

3. 设置有效期：

```
require(block.timestamp <= deadline, "Signature expired");
```

4. 使用签名映射：

记录已使用的签名，防止重复

```
mapping(bytes32 => bool) public usedSignatures;

bytes32 sigHash = keccak256(signature);
require(!usedSignatures[sigHash], "Signature already used");
usedSignatures[sigHash] = true;
```

4.3 Nonce 值的选择

面试官：是否可以使用全局的 nonce 值？除了防止重放攻击，签名中还应包含哪些内容？

回答：

全局 Nonce 的问题：

不建议使用全局递增的 nonce，因为：

- 低 Gas Price 交易可能长时间 pending
- 后续交易必须等待前面的交易确认
- 降低了并发性

更好的方案：

- 按签名映射：记录每个签名的使用状态
- 按操作类型分组：不同操作使用独立的 nonce 计数器

签名应包含的内容：

1. 合约地址：防止跨合约重放
2. 链 ID：防止跨链重放
3. Nonce：防止重复使用
4. 截止时间：限制签名有效期
5. 业务数据：交易金额、接收方等
6. EIP-712 域分隔符：增强安全性

EIP-712 签名示例：

```
bytes32 DOMAIN_SEPARATOR = keccak256(abi.encode(
    keccak256("EIP712Domain(string name,string version,uint256 chainId,address
verifyingContract)"),
    keccak256(bytes("MyContract")),
    keccak256(bytes("1")),
    block.chainid,
    address(this)
));
```

4.4 交易回滚与数据一致性

面试官：是否会出现因回滚导致数据不一致的问题？

回答：

潜在问题：

理论上存在一种情况：如果交易执行失败并回滚，但 nonce 值在链下系统中已经递增，可能导致不一致。

解决方案：

1. 链上 Nonce 管理：

- Nonce 完全由智能合约管理
- 交易失败时自动回滚 nonce
- 保证原子性

2. 链下同步机制：

- 监听链上事件确认交易状态
- 失败交易的 nonce 可重用
- 维护 nonce 同步状态

3. 使用 EVM 的原子性：

```
function execute() external {  
    // 所有状态变更在同一交易中  
    nonces[msg.sender]++;  
    // ... 其他操作  
    // 任何 require 失败都会回滚所有状态  
}
```

五、代理合约安全

5.1 代理合约的核心风险

面试官：在使用代理合约时，最需要注意什么问题？

回答：

函数选择器冲突（最严重）：

代理合约和逻辑合约的函数选择器可能冲突，导致意外行为。

示例：

```
// 代理合约  
contract Proxy {  
    address public implementation;  
  
    // 选择器: 0x5c60dalb  
    function implementation() public view returns (address) {  
        return implementation;  
    }  
  
    fallback() external payable {  
        // 委托调用逻辑合约  
        delegatecall(implementation);  
    }  
}
```

```
// 逻辑合约（假设也有相同选择器的函数）
contract Logic {
    // 如果这个函数的选择器也是 0x5c60da1b
    // 会导致冲突
    function someFunction() public {
        // ...
    }
}
```

其他需要注意的问题：

1. 存储槽冲突：
 - 代理合约和逻辑合约的状态变量必须布局一致
 - 使用 `delegatecall` 时，逻辑合约操作的是代理合约的存储
2. 初始化问题：
 - 逻辑合约不能使用 `constructor`
 - 需要使用 `initialize` 函数，并确保只能调用一次
3. 权限控制：
 - 必须限制 `upgradeTo` 等管理函数的调用权限
 - 防止恶意升级

最佳实践：

- 使用 OpenZeppelin 的透明代理或 UUPS 代理模式
- 使用专用的存储槽（如 EIP-1967）存储代理相关变量
- 全面的安全审计

5.2 代理合约的权限风险

面试官：安全方面，代理合约是否存在被任意调用的风险？如果不加以权限限制，会有什么后果？

回答：

风险分析：

1. 未授权升级：

```
// 危险的代码
function upgradeTo(address newImplementation) external {
    implementation = newImplementation; // 任何人都能调用！
}
```

后果：

- 攻击者可以将逻辑合约替换为恶意合约
 - 窃取所有用户资产
 - 破坏业务逻辑
2. 直接调用逻辑合约：

如果逻辑合约没有正确的访问控制，攻击者可能直接调用逻辑合约（而非通过代理），导致意外状态变更。

3. 存储槽覆盖：

攻击者通过恶意升级，覆盖关键存储槽（如 owner、余额等）

安全措施：

// 安全的代理合约示例

```
contract SecureProxy {
    address private immutable ADMIN;
    address public implementation;

    constructor(address _implementation) {
        ADMIN = msg.sender;
        implementation = _implementation;
    }

    modifier onlyAdmin() {
        require(msg.sender == ADMIN, "Not admin");
        _;
    }

    function upgradeTo(address newImplementation) external onlyAdmin {
        require(newImplementation != address(0), "Invalid address");
        require(newImplementation.code.length > 0, "Not a contract");
        implementation = newImplementation;
        emit Upgraded(newImplementation);
    }

    fallback() external payable {
        address impl = implementation;
        assembly {
            calldatacopy(0, 0, calldatasize())
            let result := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)
            returndatacopy(0, 0, returndatasize())
            switch result
            case 0 { revert(0, returndatasize()) }
            default { return(0, returndatasize()) }
        }
    }
}
```

5.3 通过代理合约窃取资金

面试官：攻击者如何通过代理合约来偷取用户的资金？

回答：

攻击场景：

1. 利用已授权的 **approve**：

```
// 用户已经 approve 给代理合约
token.approve(proxyAddress, type(uint256).max);

// 攻击者升级代理合约为恶意合约
contract MaliciousLogic {
    function stealFunds(address token, address victim) external {
        // 利用已有的 approve 转走资金
        IERC20(token).transferFrom(victim, attacker, amount);
    }
}
```

2. 利用 `delegatecall` 的特性:

- `delegatecall` 使用代理合约的存储和余额
- 恶意逻辑可以直接操作用户资产

3. 利用 `selfdestruct`:

```
contract MaliciousLogic {
    function destroy() external {
        selfdestruct(payable(attacker)); // 销毁合约, 发送所有 ETH
    }
}
```

防护措施:

1. 多签管理: 升级操作需要多个管理员签名
2. 时间锁: 升级操作有延迟期, 用户有时间撤出资产
3. 不可变代理: 某些场景使用不可升级的合约
4. 最小授权原则: 用户只授权必要的额度
5. 监报告警: 监控升级事件, 及时通知用户

六、跨链与其他技术

6.1 Solana Token 模型

面试官: Solana 链上的 Token 模型与以太坊链上的 ERC20 token 有何区别?

回答:

核心区别:

特性	Solana (SPL Token)	Ethereum (ERC20)
账户模型	账户模型 (Account Model)	账户模型
存储租金	需要支付存储租金	无租金概念
Token 账户	每个用户每种代币需要独立的 Token Account	合约内映射管理余额
创建成本	需要支付账户租金 (约 0.002 SOL)	仅 Gas 费
性能	高吞吐量 (65,000 TPS)	相对较低 (15-30 TPS)

Solana 的存储租金机制：

- 账户必须保持最低余额 (rent-exempt minimum)
- 如果余额低于最低要求，账户会被清理
- 达到最低余额后免除后续租金

SPL Token 的特点：

```
// Solana 上每个用户持有的每种代币都有独立账户
User Wallet (SOL)
├─ Token Account 1 (USDC)
├─ Token Account 2 (USDT)
└─ Token Account 3 (Custom Token)
```

6.2 EVM 交易生命周期

面试官：EVM（以太坊）虚拟机中的交易生命周期是怎样的？

回答：

完整的交易生命周期：

```
1. 交易创建
├─ 用户签名交易
├─ 指定 Gas Price 和 Gas Limit
└─ 通过 RPC 发送到节点

2. 进入内存池 (Mempool)
├─ 验证签名有效性
├─ 验证 Nonce 正确性
├─ 验证账户余额充足
└─ 按 Gas Price 排序

3. 矿工/验证者选择交易
├─ 优先选择高 Gas Price 交易
├─ 考虑 MEV (最大可提取价值)
└─ 组装成区块

4. 交易执行
```

- └ 扣除 Gas 费用
- └ 执行 EVM 字节码
 - └ 操作码解析
 - └ 栈操作
 - └ 内存操作
 - └ 存储操作
- └ 状态变更
- └ 生成收据 (Receipt)

5. 区块确认

- └ 区块被添加到链上
- └ 等待多个确认 (通常 12-20 块)
- └ 交易最终确认

EVM 执行细节:

- 将 Solidity 代码编译为字节码 (bytecode)
- EVM 解释执行字节码指令 (如 `PUSH`, `ADD`, `SSTORE`)
- 每个操作消耗特定的 Gas
- Gas 耗尽或遇到 `revert` 则交易失败回滚

七、团队与技术栈

7.1 团队技术方向

面试官: 项目组主要负责区块链安全和合约开发吗?

回答:

是的, 项目组的核心职责包括:

- 智能合约的设计与开发
- 区块链安全审计与防护
- 业务逻辑开发
- 后端代码审核
- 系统安全性保障

7.2 技术栈

面试官: 部门主要使用哪种编程语言编写智能合约?

回答:

部门主要使用的智能合约开发语言:

- **Solidity**: 用于 EVM 兼容链 (以太坊、BSC、Polygon 等)
- **Rust**: 用于 Solana、Near 等高性能链

7.3 岗位要求

面试官： 对于求职者来说，是需要具备较强的区块链后端服务技术栈吗？

回答：

技能要求：

合约开发岗：

- 必须熟悉区块链原理和智能合约开发
- 掌握 Solidity/Rust 等合约语言
- 了解常见安全漏洞和防护措施
- 具备合约审计能力

后端服务岗：

- 具备扎实的后端开发能力
- 了解区块链基础知识
- 熟悉链上数据同步、钱包服务等
- 根据团队需求可能参与合约相关工作

复合型人才：

团队欢迎既懂合约又懂后端的复合型人才，可以在不同方向发挥所长。